Penjelasan Koding Backend — MiniProjectKPI_EWI

Dokumen ini menjelaskan isi file-file Python di folder `backend/` dengan bahasa sederhana. Fokus hanya pada **file yang dipakai di produksi** (bukan script ad-hoc).

---

1. File: `config.py` (20 baris)

File ini menyimpan **semua pengaturan aplikasi**.

No	Tipe	Nama	Penjelasan Sederhana
1	Variable	`BASE_DIR`	Lokasi folder backend di komputer
2	**Class**	`Config`	Kumpulan semua pengaturan aplikasi
3	Variable	`SECRET_KEY`	Kunci rahasia untuk keamanan session Flask
4	Variable	`SQLALCHEMY_DATABASE_URI`	Lokasi database SQLite (`database.db`)
5	Variable	`JWT_SECRET_KEY`	Kunci rahasia untuk token login (JWT)
6	Variable	`UPLOAD_FOLDER`	Folder tempat simpan file bukti/evidence
7	Variable	`EXPORT_FOLDER`	Folder output laporan export
8	Variable	`REPORT_DEFAULT_YEAR`	Tahun default laporan (2024)
9	Variable	`MAX_CONTENT_LENGTH`	Batas ukuran upload (16 MB)
10	Variable	`ALLOWED_EXTENSIONS`	Tipe file yang boleh diupload (png, jpg, pdf, dll)

Ringkas: 20 baris | 1 Class | 0 Function

---

2. File: `models.py` (322 baris) ★

File ini mendefinisikan **struktur semua tabel database**. Seperti template Excel untuk setiap jenis data.

Class `User` (baris 8–33)

Template data pengguna sistem.

No	Tipe	Nama	Penjelasan Sederhana
1	Kolom	`username`, `password_hash`, `full_name`, `role`	Data dasar user
2	def	`set_password(password)`	Enkripsi password sebelum disimpan
3	def	`check_password(password)`	Cek password saat login

No	Tipe	Nama	Penjelasan Sederhana
4	def	`to_dict()`	Ubah data user jadi format JSON

Class `Category` (baris 36–59)

Template kategori pengeluaran, mendukung **parent-child** (bertingkat).

No	Tipe	Nama	Penjelasan Sederhana
1	Kolom	`name`, `code`, `parent_id`, `status`, `created_by`	Data kategori
2	def	`to_dict(include_children)`	Ubah ke JSON, opsi sertakan sub-kategori

Class `Advance` (baris 62–107)

Template kasbon (pengajuan dana di muka).

No	Tipe	Nama	Penjelasan Sederhana
1	Kolom	`title`, `description`, `user_id`, `status`, `notes`, `approved_at`, `advance_type`	Data kasbon
2	def	`total_amount()`	Hitung total semua item kasbon
3	def	`approved_amount()`	Hitung total yang disetujui
4	def	`to_dict(include_items)`	Ubah ke JSON, opsi sertakan item

Class `AdvanceItem` (baris 110–135)

Item rencana biaya di dalam kasbon.

No	Tipe	Nama	Penjelasan Sederhana
1	Kolom	`advance_id`, `category_id`, `description`, `estimated_amount`, `evidence_path`	Data item
2	def	`to_dict()`	Ubah ke JSON

Class `Settlement` (baris 138–183)

Template realisasi pengeluaran.

No	Tipe	Nama	Penjelasan Sederhana
1	Kolom	`title`, `description`, `user_id`, `settlement_type`, `status`, `advance_id`	Data settlement
2	def	`total_amount()`	Hitung total semua expense
3	def	`approved_amount()`	Hitung total expense disetujui
4	def	`to_dict(include_expenses)`	Ubah ke JSON

Class `Expense` (baris 186–229)

Detail nota pengeluaran aktual.

No	Tipe	Nama	Penjelasan Sederhana
1	Kolom	`settlement_id`, `category_id`, `description`, `amount`, `date`, `source`, `currency`, `currency_exchange`, `status`, `notes`	Data expense
2	def	`idr_amount()`	Hitung jumlah dalam IDR (kalau mata uang asing, dikalikan kurs)
3	def	`to_dict()`	Ubah ke JSON

Class `Revenue` (baris 232–281)

Pencatatan pemasukan/pendapatan.

No	Tipe	Nama	Penjelasan Sederhana
1	Kolom	`invoice_date`, `description`, `invoice_value`, `currency`, `client`, `ppn`, `pph_23`, dll	Data revenue
2	def	`idr_invoice_value()`	Nilai faktur dalam IDR
3	def	`idr_amount_received()`	Nilai diterima dalam IDR
4	def	`to_dict()`	Ubah ke JSON

Class `Tax` (baris 284–320)

Pencatatan pajak.

No	Tipe	Nama	Penjelasan Sederhana
1	Kolom	`date`, `description`, `transaction_value`, `currency`, `ppn`, `pph_21`, `pph_23`, `pph_26`	Data pajak
2	def	`idr_transaction_value()`	Nilai transaksi dalam IDR
3	def	`to_dict()`	Ubah ke JSON

Ringkas: 322 baris | 7 Class | 17 Function (termasuk property)

---

3. File: `app.py` (487 baris)

File ini adalah **titik mulai aplikasi** — inisialisasi Flask, register semua route, dan isi data awal.

No	Tipe	Nama	Penjelasan Sederhana
1	def	`create_app()`	**Nyalakan aplikasi** — buat Flask, pasang JWT, CORS, register semua blueprint route
2	def	`serve_upload(filename)`	**Kirim file upload** — endpoint publik untuk akses file evidence
3	def	`_extract_sqlite_path(uri)`	Helper: ambil path file `.db` dari URI SQLAlchemy
4	def	`_looks_hashed(password_hash)`	Helper: cek apakah password sudah dienkripsi
5	def	`ensure_advance_type_column(app)`	**Migrasi otomatis** — tambah kolom `advance_type` jika belum ada di DB lama
6	def	`bootstrap_from_database_new(app)`	**Impor data** — copy data dari `database_new.db` ke `database.db` jika DB utama kosong
7	def	`seed_data()`	**Isi data default** — buat user, kategori, revenue, tax awal jika DB kosong

Blueprint yang didaftarkan:

Prefix	Modul
`/api/auth`	`auth.py`
`/api/settlements`	`settlements.py`
`/api/expenses`	`expenses.py`
`/api/reports`	`reports.py`
`/api/categories`	`categories.py`
`/api/advances`	`advances.py`
`/api/settings`	`settings.py`
`/api/revenues`	`revenues.py`
`/api/taxes`	`taxes.py`

Ringkas: 487 baris | 0 Class | 7 Function

4. File: `routes/auth.py` (74 baris)

Mengatur **login, profil, dan manajemen user**.

No	Tipe	Nama	Route	Penjelasan Sederhana
1	def	`login()`	`POST /api/auth/login`	Proses login — terima username+password, kembalikan token JWT
2	def	`me()`	`GET /api/auth/me`	Ambil profil user yang sedang login
3	def	`list_users()`	`GET /api/auth/users`	List semua user (hanya manager)
4	def	`create_user()`	`POST /api/auth/users`	Buat user baru (hanya manager)

Ringkas: 74 baris | 0 Class | 4 Function

---

5. File: `routes/settlements.py` (288 baris)

Mengatur **CRUD dan workflow settlement** (realisasi pengeluaran).

No	Tipe	Nama	Route	Penjelasan Sederhana
1	def	`_has_any_report_tags()`	*(internal)*	Cek apakah tabel `report_entry_tags` punya data
2	def	`_tagged_settlement_ids_for_year(year)`	*(internal)*	Ambil ID settlement yang di-tag untuk tahun tertentu
3	def	`list_settlements()`	`GET /api/settlements`	List settlement + filter status/tipe/tahun/tanggal
4	def	`create_settlement()`	`POST /api/settlements`	Buat settlement baru (single/batch), bisa link ke Advance
5	def	`get_settlement(id)`	`GET /api/settlements/<id>`	Detail 1 settlement + semua expense-nya
6	def	`update_settlement(id)`	`PUT /api/settlements/<id>`	Edit settlement (hanya owner, status draft/rejected)
7	def	`delete_settlement(id)`	`DELETE /api/settlements/<id>`	Hapus settlement draft
8	def	`submit_settlement(id)`	`POST /api/settlements/<id>/submit`	Submit settlement untuk di-review manager

No	Tipe	Nama	Route	Penjelasan Sederhana
9	def	`approve_all_expenses(id)`	`POST /api/settlements/<id>/approve_all`	Manager approve semua expense sekaligus
10	def	`reject_all_expenses(id)`	`POST /api/settlements/<id>/reject_all`	Manager reject semua expense + catatan alasan

Ringkas: 288 baris | 0 Class | 10 Function

---

6. File: `routes/expenses.py` (231 baris)

Mengatur **CRUD expense** (nota pengeluaran) termasuk upload bukti.

No	Tipe	Nama	Route	Penjelasan Sederhana
1	def	`allowed_file(filename)`	*(internal)*	Cek apakah ekstensi file diizinkan
2	def	`create_expense()`	`POST /api/expenses`	Tambah expense baru + upload file evidence (multipart)
3	def	`update_expense(id)`	`PUT /api/expenses/<id>`	Edit expense — status otomatis reset ke `pending`
4	def	`delete_expense(id)`	`DELETE /api/expenses/<id>`	Hapus expense + file evidence-nya
5	def	`approve_expense(id)`	`POST /api/expenses/<id>/approve`	Manager approve expense
6	def	`reject_expense(id)`	`POST /api/expenses/<id>/reject`	Manager reject expense + catatan alasan
7	def	`serve_evidence(filename)`	`GET /api/expenses/evidence/<path>`	Kirim file evidence untuk dilihat/download
8	def	`list_categories()`	`GET /api/expenses/categories`	List kategori yang sudah approved

Ringkas: 231 baris | 0 Class | 8 Function

---

7. File: `routes/advances.py` (328 baris)

Mengatur **CRUD dan workflow kasbon** (advance/pengajuan dana).

CRUD Advance (Header)

No	Tipe	Nama	Route	Penjelasan Sederhana
1	def	`allowed_file(filename)`	*(internal)*	Cek ekstensi file
2	def	`list_advances()`	`GET /api/advances`	List kasbon + filter status/tanggal
3	def	`create_advance()`	`POST /api/advances`	Buat kasbon baru
4	def	`get_advance(id)`	`GET /api/advances/<id>`	Detail 1 kasbon + semua item-nya
5	def	`update_advance(id)`	`PUT /api/advances/<id>`	Edit kasbon (hanya owner, status draft/rejected)
6	def	`delete_advance(id)`	`DELETE /api/advances/<id>`	Hapus kasbon draft

CRUD Advance Items

No	Tipe	Nama	Route	Penjelasan Sederhana
7	def	`add_advance_item(id)`	`POST /api/advances/<id>/items`	Tambah item kasbon + upload evidence
8	def	`update_advance_item(id)`	`PUT /api/advances/items/<id>`	Edit item + ganti evidence
9	def	`delete_advance_item(id)`	`DELETE /api/advances/items/<id>`	Hapus item + file evidence

Workflow

No	Tipe	Nama	Route	Penjelasan Sederhana
10	def	`submit_advance(id)`	`POST /api/advances/<id>/submit`	Submit kasbon untuk review manager
11	def	`approve_advance(id)`	`POST /api/advances/<id>/approve_all`	Manager approve kasbon
12	def	`reject_advance(id)`	`POST /api/advances/<id>/reject_all`	Manager reject kasbon + catatan

Ringkas: 328 baris | 0 Class | 12 Function

---

8. File: `routes/categories.py` (158 baris)

Mengatur **manajemen kategori** pengeluaran (parent-child).

No	Tipe	Nama	Route	Penjelasan Sederhana
1	def	`list_categories()`	`GET /api/categories`	List kategori — staff hanya lihat approved, manager lihat semua
2	def	`list_pending()`	`GET /api/categories/pending`	List kategori pending (hanya manager)
3	def	`create_category()`	`POST /api/categories`	Buat kategori — staff → status pending, manager → langsung approved
4	def	`update_category(id)`	`PUT /api/categories/<id>`	Edit nama/parent kategori (hanya manager)
5	def	`delete_category(id)`	`DELETE /api/categories/<id>`	Hapus kategori (hanya jika tidak ada expense terkait)
6	def	`approve_category(id)`	`POST /api/categories/<id>/approve`	Manager approve/reject kategori pending

Ringkas: 158 baris | 0 Class | 6 Function

---

9. File: `routes/revenues.py` (146 baris)

Mengatur **CRUD data pemasukan/pendapatan** (revenue).

No	Tipe	Nama	Route	Penjelasan Sederhana
1	def	`_parse_date(date_str)`	*(internal)*	Helper: parse string tanggal ke objek date
2	def	`get_revenues()`	`GET /api/revenues`	List revenue + filter tanggal
3	def	`get_revenue(id)`	`GET /api/revenues/<id>`	Detail 1 revenue
4	def	`create_revenue()`	`POST /api/revenues`	Buat revenue baru (hanya manager)
5	def	`update_revenue(id)`	`PUT /api/revenues/<id>`	Edit revenue (hanya manager)
6	def	`delete_revenue(id)`	`DELETE /api/revenues/<id>`	Hapus revenue (hanya manager)

Ringkas: 146 baris | 0 Class | 6 Function

---

10. File: `routes/taxes.py` (133 baris)

Mengatur **CRUD data pajak** (tax).

No	Tipe	Nama	Route	Penjelasan Sederhana
1	def	`_parse_date(date_str)`	*(internal)*	Helper: parse string tanggal ke objek date
2	def	`get_taxes()`	`GET /api/taxes`	List pajak + filter tanggal
3	def	`get_tax(id)`	`GET /api/taxes/<id>`	Detail 1 pajak
4	def	`create_tax()`	`POST /api/taxes`	Buat data pajak baru (hanya manager)
5	def	`update_tax(id)`	`PUT /api/taxes/<id>`	Edit data pajak (hanya manager)
6	def	`delete_tax(id)`	`DELETE /api/taxes/<id>`	Hapus data pajak (hanya manager)

Ringkas: 133 baris | 0 Class | 6 Function

---

11. File: `routes/settings.py` (128 baris)

Mengatur **pengaturan aplikasi** (storage & tahun laporan).

No	Tipe	Nama	Route	Penjelasan Sederhana
1	def	`manage_storage()`	`GET/POST /api/settings/storage`	**GET**: lihat folder storage aktif. **POST**: pindah folder storage + copy semua file + update `.env`
2	def	`manage_report_year()`	`GET/POST /api/settings/report-year`	**GET**: baca tahun laporan default. **POST**: ubah tahun default + simpan ke `.env`

Ringkas: 128 baris | 0 Class | 2 Function

---

12. File: `routes/reports.py` (1977 baris) ★ FILE TERBESAR & TERKOMPLEKS

File ini mengatur **semua ekspor laporan** (Excel + PDF). Berisi 56 function.

🔧 Helper Umum

No	Tipe	Nama	Penjelasan Sederhana
1	def	`_default_report_year()`	Ambil tahun laporan default dari config
2	def	`_safe_set_cell(ws, row, col, value)`	Tulis ke sel Excel, **SKIP jika merged cell**
3	def	`_safe_set_number(ws, row, col, value)`	Sama tapi paksa format angka (hindari auto-format tanggal)
4	def	`_normalize_external_formula_refs(wb)`	Perbaiki formula Excel yang merujuk ke file eksternal
5	def	`_clear_range(ws, r1, r2, c1, c2)`	Kosongkan rentang sel (data + formula)
6	def	`_clear_data_keep_formulas(ws, ...)`	Kosongkan data tapi **pertahankan formula**
7	def	`_set_rows_hidden(ws, r1, r2, hidden)`	Sembunyikan/tampilkan baris

📍 Helper Mapping & Parsing

No	Tipe	Nama	Penjelasan Sederhana
8	def	`_extract_imported_row(notes)`	Baca “Imported from row X” dari kolom notes
9	def	`_extract_imported_sheet_row(text)`	Baca “Imported from Sheet1 row X” dari description
10	def	`_is_date_like(value)`	Cek apakah suatu nilai berbentuk tanggal
11	def	`_is_template_detail_data_row(ws, row)`	Cek apakah baris template berisi data (punya nomor urut)
12	def	`_map_expense_category_index(expense)`	Mapping expense → index kolom template (pakai objek Expense)
13	def	`_map_expense_category_index_from_name(name)`	Mapping nama kategori → index kolom template (pakai keyword)
14	def	`_pick_template_formula_col(ws, row)`	Cari kolom mana di template yang punya formula
15	def	`_get_expense_blocks(ws)`	Temukan blok “Expense#N” di template
16	def	`_parse_iso_date(date_str)`	Parse tanggal ISO ke objek date

Export Summary (Laporan Ringkasan)

No	Tipe	Nama	Route	Penjelasan Sederhana
17	def	`_get_summary_approved_expenses(year, ...)`	*(internal)*	Ambil expense approved dari DB, filter tahun/tanggal
18	def	`_build_summary_payload(expenses)`	*(internal)*	Buat data ringkasan per kategori per bulan
19	def	`get_summary_report()`	`GET /api/reports/summary`	Kirim data summary sebagai JSON
20	def	`export_summary_pdf()`	`GET /api/reports/summary/pdf`	Export summary → PDF (ReportLab)
21	def	`generate_excel_report()`	`GET /api/reports/excel`	Export summary → Excel (dari nol, kolom kategori dinamis)

Export Kasbon

No	Tipe	Nama	Route	Penjelasan Sederhana
22	def	`generate_excel_advance_report()`	`GET /api/reports/excel_advance`	Export kasbon → Excel (dari nol)
23	def	`generate_pdf_advance_report(id)`	`GET /api/reports/advance/<id>/pdf`	Export 1 kasbon → PDF

Export Settlement & Bulk

No	Tipe	Nama	Route	Penjelasan Sederhana
24	def	`generate_settlement_receipt(id)`	`GET /api/reports/settlement/<id>/receipt`	Export 1 settlement → PDF (kuitansi)
25	def	`generate_bulk_settlements_pdf()`	`GET /api/reports/settlements/pdf`	Export banyak settlement → 1 PDF
26	def	`generate_bulk_advances_pdf()`	`GET /api/reports/advances/pdf`	Export banyak kasbon → 1 PDF

Export Annual (Laporan Tahunan) — Paling Kompleks ⚠

Helper Annual

No	Tipe	Nama	Penjelasan Sederhana
27	def	`_is_true(value)`	Cek flag boolean (string/int)
28	def	`_annual_cache_paths(year)`	Path file cache annual (JSON + PDF)

No	Tipe	Nama	Penjelasan Sederhana
29	def	`_load_annual_payload_cache(year)`	Baca cache data annual dari file JSON
30	def	`_save_annual_payload_cache(year, data)`	Simpan cache data annual ke file JSON
31	def	`_to_float(value)`	Konversi ke float (aman dari error)
32	def	`_safe_text(value)`	Konversi ke string (aman dari None)
33	def	`_as_iso_date(value)`	Format tanggal ke ISO string
34	def	`_idr_from_currency(amount, currency, exchange)`	Hitung nilai IDR dari mata uang asing
35	def	`_shorten(text, size)`	Potong teks panjang + "..."
36	def	`_map_expense_column(category_name)`	Mapping nama kategori → kolom Excel template
37	def	`_extract_batch_number(text)`	Ambil nomor batch dari title settlement
38	def	`_is_batch_settlement(type, title)`	Cek apakah settlement bertipe batch
39	def	`_clean_settlement_title(title)`	Bersihkan prefix "Single:", "Batch:" dari judul
40	def	`_group_annual_expenses(expenses, year)`	Kelompokkan expense per settlement, urutkan by tanggal
41	def	`_tagged_ids_for_year(table, year)`	Ambil ID yang di-tag untuk tahun tertentu (dari `report_entry_tags`)
42	def	`_has_any_report_tags()`	Cek apakah tabel `report_entry_tags` punya data

Pembangun Data Annual

No	Tipe	Nama	Penjelasan Sederhana
43	def	`get_annual_report()`	`GET /api/reports/annual` — Kirim data annual sebagai JSON (pakai cache)
44	def	`_build_annual_payload_from_db(year)`	**Query semua data** (revenue, tax, expense) dari DB, buat payload lengkap
45	def	`_root_category_name(category_id)`	Naik dari child category ke root parent-nya
46	def	`_build_annual_pdf_bytes(payload)`	Render data annual → file PDF bytes (ReportLab)

Penulis Sheet Sekunder

No	Tipe	Nama	Penjelasan Sederhana
47	def	`_write_secondary_summary_sheets(wb, payload, year)`	Tulis sheet “Laba Rugi” dan “Business Summary” dari data
48	def	`_clone_sheet_from_template(src, target, name)`	Copy sheet dari template donor ke workbook output
49	def	`_ensure_formatted_secondary_sheets(wb, year, dir)`	Pastikan sheet sekunder ada (copy dari template atau buat baru)
50	def	`_save_annual_pdf_cache(year, pdf_bytes)`	Simpan file PDF annual ke cache

Endpoint Annual

No	Tipe	Nama	Route	Penjelasan Sederhana
51	def	`get_annual_report_pdf()`	`GET /api/reports/annual/pdf`	Export annual → PDF (pakai cache)
52	def	`get_annual_report_excel()`	`GET /api/reports/annual/excel`	⭐ **FUNGSI TERBESAR** — Buka template Excel, isi data revenue/tax/expense, kirim file .xlsx

Cara Kerja `get_annual_report_excel()` (baris 1593–1975):

```
1. Baca data dari DB (revenue, tax, expense)
2. Buka file template Excel dari folder excel/
3. Perbaiki formula referensi eksternal
4. Tulis data Revenue ke baris 8–21
5. Tulis data Tax ke baris 27–36
6. Tulis expense Single ke baris 41–96 (Tabel 3)
7. Tulis expense Batch ke blok Expense#N
8. Pastikan sheet sekunder ada (Laba Rugi, Business Summary)
9. Simpan workbook → kirim ke user
```

Fungsi Penulisan Expense ke Template

No	Tipe	Nama	Penjelasan Sederhana
53	def	`_write_expense_line(ws, row, seq, expense)`	Tulis 1 baris expense ke template (summary/single)
54	def	`_write_expense_detail_line(ws, row, seq, expense)`	Tulis 1 baris expense ke blok batch detail

Ringkas: 1977 baris | 0 Class | 56 Function

---

13. File: `migrate.py` (37 baris)

Script CLI sederhana untuk **migrasi database** menggunakan Flask-Migrate (Alembic).

No	Tipe	Nama	Penjelasan Sederhana
1	Top-Level	*(menu interaktif)*	Pilih opsi: (1) init folder migrasi, (2) buat file migrasi baru, (3) terapkan migrasi ke DB

> ⚠️ Dipakai saat ada perubahan di `models.py` — jalankan manual, **bukan bagian runtime**.

Ringkas: 37 baris | 0 Class | 0 Function

---

14. File: `migrate_evidence.py` (25 baris)

Script untuk **menambah kolom** `evidence_path` dan `evidence_filename` ke tabel `advance_items`.

No	Tipe	Nama	Penjelasan Sederhana
1	def	`upgrade()`	Cek apakah kolom sudah ada → jika belum, `ALTER TABLE` untuk menambahkannya

> ⚠️ Dijalankan 1x saja saat upgrade skema DB lama. **Bukan bagian runtime**.

Ringkas: 25 baris | 0 Class | 1 Function

---

15. File: `scripts/excel_to_app_db.py` (1321 baris) ⭐ SCRIPT IMPOR DATA

Script besar untuk **konversi file Excel template ke database SQLite** yang siap dipakai aplikasi.

Konstanta & Mapping Penting

No	Tipe	Nama	Penjelasan Sederhana
1	Variable	`REVENUE_ROW_START/END`	Baris 8–21 = area Revenue di template
2	Variable	`TAX_ROW_START/END`	Baris 27–36 = area Tax di template
3	Variable	`STANDARD_SUBCATEGORIES`	13 nama subkategori standar (Transportation, Accommodation, dll)
4	Variable	`STANDARD_SUBCATEGORY_TO_PARENT_CODE`	Mapping subkategori → kode parent (misal Transportation → “A” = Biaya Operasi)
5	Variable	`FULL_MAPPING`	Mapping 40+ variasi teks typo/legacy → nama standar

Helper Konversi Data

No	Tipe	Nama	Penjelasan Sederhana
6	def	`normalize_subcategory(desc)`	Normalisasi teks typo ke 13 nama standar
7	def	`parse_args()`	Parsing argumen CLI (`--excel`, `--output-db`, dll)
8	def	`default_output_dir()`	Path default folder output (`data/`)
9	def	`build_output_db_path(...)`	Tentukan path file DB output
10	def	`to_iso_datetime(value)`	Konversi apapun → format datetime ISO
11	def	`to_iso_date(value)`	Konversi apapun → format date ISO
12	def	`to_num(value)`	Konversi apapun → float (aman dari error)
13	def	`_eval_formula_expr(expr, ...)`	Evaluasi formula Excel sederhana (misal `=7*100`)
14	def	`to_num_cell(ws, row, col)`	Baca angka dari sel Excel, fallback ke formula jika cached value kosong
15	def	`to_text(value)`	Konversi apapun → string
16	def	`build_subcategory_alias_map()`	Buat mapping lowercase → nama standar

Fungsi Database & Skema

No	Tipe	Nama	Penjelasan Sederhana
17	def	`ensure_schema(conn, ...)`	Buat semua tabel SQL jika belum ada (users, categories, settlements, dll)
18	def	`tag_report_row(conn, table, row_id, year, ...)`	Tag entry di `report_entry_tags` untuk tracking tahun data
19	def	`ensure_reference_data(conn, ...)`	Buat user default + 9 kategori parent + subkategorinya jika DB kosong
20	def	`purge_year_data(conn, year)`	Hapus data tahun tertentu sebelum re-import (by tag atau by tanggal)
21	def	`store_excel_structure(conn, ...)`	Simpan metadata sel Excel ke tabel `excel_cells` (opsional)
22	def	`extract_year_from_header(ws)`	Baca tahun dari header Excel (baris 2, kolom 4)
23	def	`tag_cell_mapping(conn, ...)`	Update mapping sel Excel → tabel + kolom target di DB

Fungsi Impor Data

No	Tipe	Nama	Penjelasan Sederhana
24	def	`import_revenues(conn, ...)`	Impor data Revenue dari baris 8–21 template ke tabel `revenues`
25	def	`import_taxes(conn, ...)`	Impor data Tax dari baris 27–36 template ke tabel `taxes`
26	def	`build_expense_blocks(ws, ...)`	Scan template → temukan baris mandiri (single) dan blok `Expense#N` (batch)
27	def	`detect_category_id(ws, row, ...)`	Tentukan kategori expense berdasarkan subkategori teks + kolom mana yang ada angkanya
28	def	`import_expenses(conn, ...)`	⭐ **Fungsi utama impor** — buat settlement + expense dari baris mandiri dan blok batch

Entry Point

No	Tipe	Nama	Penjelasan Sederhana
29	def	`main()`	Buka Excel → buat/bersihkan DB → impor revenue, tax, expense → cetak ringkasan

> ⚠️ Jalankan manual: `python backend/scripts/excel_to_app_db.py --excel "path/to/file.xlsx"`

Ringkas: 1321 baris | 0 Class | 29 Function

---

16. File: `scripts/clean_subcategory.py` (146 baris)

Script untuk **normalisasi label subkategori legacy** dalam file Excel sebelum di-impor.

No	Tipe	Nama	Penjelasan Sederhana
1	Variable	`MAPPING`	40+ mapping teks legacy → nama standar (sama isi dengan `FULL_MAPPING` di `excel_to_app_db.py`)
2	def	`parse_args()`	Parse argumen CLI (`--input`, `--output`, `--sheet`)
3	def	`clean_subcategories(input, output, sheet)`	Buka Excel → scan blok Expense# → ganti label subkategori typo → simpan file baru
4	def	`main()`	Entry point script

> ⚠️ Dipakai sebelum impor data Excel yang label subkategorinya belum rapi. **Bukan runtime**.

Ringkas: 146 baris | 0 Class | 3 Function

---

17. File: `scripts/verify_subcategories.py` (61 baris)

Script audit untuk **melihat daftar subkategori unik** yang ada di file Excel.

No	Tipe	Nama	Penjelasan Sederhana
1	Top-Level	*(script langsung)*	Buka Excel → cari header `Expense#N` → scan baris tanpa tanggal/amount → cetak semua subkategori unik

⚠️ Dipakai untuk audit/debugging mapping subkategori. **Bukan runtime**.

Ringkas: 61 baris | 0 Class | 0 Function

Ringkasan Total Backend

File	Baris	Class	Function	Fungsi Utama
`config.py`	20	1	0	Pengaturan aplikasi
`models.py`	322	7	17	⭐ Struktur semua tabel database
`app.py`	487	0	7	Inisialisasi Flask + seed data
`routes/auth.py`	74	0	4	Login & manajemen user
`routes/settlements.py`	288	0	10	CRUD & workflow settlement
`routes/expenses.py`	231	0	8	CRUD expense + upload evidence
`routes/advances.py`	328	0	12	CRUD & workflow kasbon
`routes/categories.py`	158	0	6	Manajemen kategori
`routes/revenues.py`	146	0	6	CRUD revenue
`routes/taxes.py`	133	0	6	CRUD pajak
`routes/settings.py`	128	0	2	Settings storage & tahun
`routes/reports.py`	1977	0	56	⭐ Semua export laporan
`migrate.py`	37	0	0	Migrasi skema DB (manual)
`migrate_evidence.py`	25	0	1	Tambah kolom evidence (1x)
`scripts/excel_to_app_db.py`	1321	0	29	⭐ Impor Excel → DB
`scripts/clean_subcategory.py`	146	0	3	Normalisasi subkategori
`scripts/verify_subcategories.py`	61	0	0	Audit subkategori
TOTAL	**5882**	**8**	**167**	

Catatan Penting

1. **File runtime utama (1–12):** Dipakai saat server berjalan — ini yang paling penting dipahami pertama
2. **File utilitas (13–17):** Dipakai manual saat maintenance/impor data — tidak berjalan otomatis
3. **Fungsi terbesar/terkompleks:** `get_annual_report_excel()` di `reports.py` — 380+ baris
4. **Database:** SQLite — file tunggal `database.db`
5. **Autentikasi:** JWT (JSON Web Token) — semua endpoint kecuali login butuh token
6. **Hak akses:** Manager bisa CRUD semua + approve/reject; Staff/Mitra hanya bisa CRUD milik sendiri
7. **File yang TIDAK didokumentasi** (script ad-hoc/experiment): `read_excel.py`, `read_excel2.py`, `temp_check.py`, `temp_inspect.py`, `update_categories.py`, `update_excel_db.py`